Module 04: Svelte 5 Core Concepts

Master the runes, reactivity system, and component patterns that make Svelte 5 unique.

Revolution Trading Pros - Web Dev Course

Overview

Svelte 5 introduces a revolutionary reactivity system based on 'runes' - special function-like primitives that tell the Svelte compiler how to handle reactivity. This module covers every core concept you need to build interactive Svelte 5 applications.

If you have used earlier versions of Svelte, forget what you knew about $: reactive declarations and stores. Runes are simpler, more explicit, and more powerful.

Components and .svelte Files

A Svelte component is a self-contained unit of UI written in a .svelte file. Each component can contain three sections: a script block for logic, markup for structure, and a style block for presentation.

Svelte

```
<script lang="ts">
  // Logic goes here
  let greeting = "Hello";
</script>

<!-- Markup (HTML) goes here -->
<h1>{greeting}, World!</h1>

<style>
  /* Scoped styles go here */
  h1 {
    color: #ff3e00;
  }
</style>
```

The script block uses lang="ts" for TypeScript support (recommended). The markup can include dynamic expressions in curly braces. Styles are automatically scoped to this component.

The $state() Rune

The $state() rune creates reactive state. When state changes, Svelte automatically updates the DOM. This is the most fundamental rune and the one you will use most often.

Svelte

```
<script lang="ts">
  let count = $state(0);
  let name = $state("World");
  let items = $state<string[]>([]);

  // Object state
  let user = $state({
    name: "Alice",
    email: "alice@example.com",
    preferences: { theme: "dark" }
  });

  function increment() {
    count++;  // Direct mutation triggers updates
  }

  function addItem(item: string) {
    items.push(item);  // Array mutations work too!
  }

  function updateTheme(theme: string) {
    user.preferences.theme = theme;  // Deep mutations are reactive
  }
</script>

<button onclick={increment}>Count: {count}</button>
<p>Hello, {name}!</p>
```

Key insight: In Svelte 5, you can directly mutate state (push, splice, property assignment) and the UI will update. You do not need to reassign the entire variable like in React or previous Svelte versions.

State with Classes

Svelte

```
<script lang="ts">
  class Counter {
    value = $state(0);

    increment() {
      this.value++;
    }

    reset() {
      this.value = 0;
    }
  }

  const counter = new Counter();
</script>

<button onclick={() => counter.increment()}>
  Count: {counter.value}
</button>
```

The $derived() Rune

The $derived() rune creates computed values that automatically update when their dependencies change. Think of it as a formula in a spreadsheet.

Svelte

```
<script lang="ts">
  let price = $state(100);
  let quantity = $state(2);
  let taxRate = $state(0.1);

  // Automatically recalculates when price, quantity, or taxRate changes
  let subtotal = $derived(price * quantity);
  let tax = $derived(subtotal * taxRate);
  let total = $derived(subtotal + tax);

  // Derived from arrays
  let items = $state([5, 10, 15, 20]);
  let sum = $derived(items.reduce((a, b) => a + b, 0));
  let average = $derived(sum / items.length);

  // String derived values
  let firstName = $state("John");
  let lastName = $state("Doe");
  let fullName = $derived(`${firstName} ${lastName}`);
</script>

<p>Subtotal: ${subtotal}</p>
<p>Tax: ${tax.toFixed(2)}</p>
<p>Total: ${total.toFixed(2)}</p>
```

$derived.by() for Complex Computations

When you need more than a single expression, use $derived.by() which takes a function:

Svelte

```
<script lang="ts">
  let items = $state([
    { name: "Apple", price: 1.50, quantity: 3 },
    { name: "Banana", price: 0.75, quantity: 6 }
  ]);

  let summary = $derived.by(() => {
    const total = items.reduce((sum, item) => {
      return sum + item.price * item.quantity;
    }, 0);
    const count = items.reduce((sum, item) => sum + item.quantity, 0);
    return { total, count, average: total / count };
  });
</script>

<p>Total: ${summary.total.toFixed(2)} for {summary.count} items</p>
```

The $effect() Rune

The $effect() rune runs side effects when reactive dependencies change. It automatically tracks which state values you read and re-runs when they change. Effects run after the DOM has updated.

Svelte

```
<script lang="ts">
  let count = $state(0);
  let query = $state("");

  // Runs whenever count changes
  $effect(() => {
    console.log(`Count is now: ${count}`);
  });

  // Debounced search effect
  $effect(() => {
    const trimmed = query.trim();
    if (trimmed.length < 3) return;

    const timeout = setTimeout(async () => {
      const response = await fetch(`/api/search?q=${trimmed}`);
      // handle response...
    }, 300);

    // Cleanup function - runs before re-executing
    return () => clearTimeout(timeout);
  });

  // DOM manipulation effect
  $effect(() => {
    document.title = `Count: ${count}`;
  });
</script>
```

Important: $effect() should be used sparingly. Prefer $derived() for computed values. Use $effect() only for side effects like DOM manipulation, API calls, logging, or integrating with third-party libraries.

The $props() Rune

The $props() rune receives properties passed to a component from its parent. It replaces the old 'export let' syntax and provides full TypeScript support.

Svelte

```
<!-- UserCard.svelte -->
<script lang="ts">
  interface Props {
    name: string;
    email: string;
    avatar?: string;          // Optional
    role?: "admin" | "user";  // Optional with union type
  }

  let { name, email, avatar = "/default-avatar.png", role = "user" } =
$props<Props>();
</script>

<div class="card">
  <img src={avatar} alt={name} />
  <h3>{name}</h3>
  <p>{email}</p>
  <span class="badge">{role}</span>
</div>
```

Svelte

```
<!-- Parent.svelte - Using the component -->
<script lang="ts">
  import UserCard from "$lib/components/UserCard.svelte";
</script>

<UserCard name="Alice" email="alice@example.com" role="admin" />
<UserCard name="Bob" email="bob@example.com" />
```

Rest Props

Svelte

```
<script lang="ts">
  let { href, children, ...rest } = $props<{
    href: string;
    children: any;
    [key: string]: any;
  }>();
</script>

<a {href} {...rest}>{@render children()}</a>
```

The $bindable() Rune

$bindable() creates props that can be two-way bound from the parent component using the bind: directive. This is useful for form inputs and interactive components.

Svelte

```
<!-- TextInput.svelte -->
<script lang="ts">
  let { value = $bindable(""), label, placeholder = "" } = $props<{
    value: string;
    label: string;
    placeholder?: string;
  }>();
</script>

<label>
  {label}
  <input type="text" bind:value {placeholder} />
</label>
```

Svelte

```
<!-- Parent.svelte -->
<script lang="ts">
  import TextInput from "$lib/components/TextInput.svelte";

  let username = $state("");
  let email = $state("");
</script>

<!-- Two-way binding: parent state updates when child input changes -->
<TextInput bind:value={username} label="Username" />
<TextInput bind:value={email} label="Email" />
<p>You entered: {username} ({email})</p>
```

Snippets and {@render}

Snippets are Svelte 5's replacement for slots. They let you define reusable chunks of markup within a component or pass markup between components.

Svelte

```
<!-- Defining and using snippets locally -->
<script lang="ts">
  let items = $state(["Apple", "Banana", "Cherry"]);
</script>

{#snippet listItem(item: string, index: number)}
  <li class="item">
    <span class="index">{index + 1}.</span>
    <span class="name">{item}</span>
  </li>
{/snippet}

<ul>
  {#each items as item, i}
    {@render listItem(item, i)}
  {/each}
</ul>
```

Passing Snippets as Props (Replacing Slots)

Svelte

```svelte
<!-- Card.svelte -->
<script lang="ts">
  import type { Snippet } from "svelte";

  let { header, children, footer }: {
    header: Snippet;
    children: Snippet;
    footer?: Snippet;
  } = $props();
</script>

<div class="card">
  <div class="card-header">{@render header()}</div>
  <div class="card-body">{@render children()}</div>
  {#if footer}
    <div class="card-footer">{@render footer()}</div>
  {/if}
</div>
```

Svelte

```svelte
<!-- Using Card -->
<Card>
  {#snippet header()}
    <h2>Card Title</h2>
  {/snippet}

  <p>This is the card body content (children).</p>

  {#snippet footer()}
    <button>Read More</button>
  {/snippet}
</Card>
```

Event Handling

Svelte 5 simplifies event handling by using standard HTML event attributes (onclick, oninput, etc.) instead of the old on: directive.

Svelte

```
<script lang="ts">
  let count = $state(0);
  let name = $state("");

  function handleClick(event: MouseEvent) {
    count++;
    console.log("Clicked at:", event.clientX, event.clientY);
  }

  function handleKeydown(event: KeyboardEvent) {
    if (event.key === "Enter") {
      console.log("Enter pressed! Name:", name);
    }
  }
</script>

<!-- Inline handler -->
<button onclick={() => count++}>Count: {count}</button>

<!-- Named handler -->
<button onclick={handleClick}>Click Me</button>

<!-- Input events -->
<input
  type="text"
  value={name}
  oninput={(e) => name = e.currentTarget.value}
  onkeydown={handleKeydown}
/>
```

Control Flow: {#if} and {#each}

Conditional Rendering

Svelte

```
<script lang="ts">
  let loggedIn = $state(false);
  let role = $state<"admin" | "user" | "guest">("guest");
</script>

{#if loggedIn}
  <p>Welcome back!</p>
  {#if role === "admin"}
    <a href="/admin">Admin Panel</a>
  {/if}
{:else}
  <p>Please log in.</p>
  <button onclick={() => loggedIn = true}>Log In</button>
{/if}
```

List Rendering

Svelte

```
<script lang="ts">
  interface Todo {
    id: number;
    text: string;
    done: boolean;
  }

  let todos = $state<Todo[]>([
    { id: 1, text: "Learn Svelte", done: false },
    { id: 2, text: "Build an app", done: false }
  ]);
</script>

{#each todos as todo (todo.id)}
  <div class="todo" class:done={todo.done}>
    <input type="checkbox" bind:checked={todo.done} />
    <span>{todo.text}</span>
  </div>
{:else}
  <p>No todos yet. Add one above!</p>
{/each}
```

Always provide a unique key expression (todo.id) in the {#each} block. This helps Svelte efficiently update the DOM when items are added, removed, or reordered.

Module 4 Exercise

Exercise: Interactive Todo List

Put what you have learned into practice with this hands-on exercise.

- Step 1: Create a new SvelteKit project or use your existing one.
- Step 2: Create a TodoList.svelte component with $state() for the todo array and input value.
- Step 3: Add an input field and 'Add' button to create new todos with unique IDs.
- Step 4: Display todos using {#each} with a key, showing checkbox and text.
- Step 5: Use $derived() to calculate completed count, remaining count, and completion percentage.
- Step 6: Add a filter (all/active/completed) using $state() and $derived().
- Step 7: Implement delete functionality with a button on each todo.
- Step 8: Add a 'Clear Completed' button that removes all done todos.
- Step 9: Use $effect() to save todos to localStorage and load them on mount.
- Step 10: Style everything with scoped CSS, including transition effects.

Summary

- $state() creates reactive state that can be directly mutated.
- $derived() computes values that update automatically when dependencies change.
- $effect() runs side effects after DOM updates; use sparingly.
- $props() receives typed component properties from parents.
- $bindable() enables two-way data binding between parent and child.
- Snippets replace slots for passing markup between components.
- Event handling uses standard HTML attributes like onclick.
- {#if} and {#each} provide declarative control flow in templates.